

Deep Learning in Practice

What to Set, and Why — before the tutorial

Prof. Benoit Lique

LMAP, Université de Pau et des Pays de l'Adour (France)

Thursday 17 September 2026

EXPOSOME ANALYTICS, Summer School, 14–18 September 2026
Imperial School of Public Health, London, United Kingdom

From the Maths to the Keyboard

This session is about the **choices you are about to make in the code**, and why they matter.

From the Maths to the Keyboard

This session is about the **choices you are about to make in the code**, and why they matter.

What the model is

- ▶ which activation, and where
- ▶ which output layer and which loss
- ▶ normalising the inputs

How it is optimised

- ▶ batch, epoch, learning rate
- ▶ overfitting, and what to watch
- ▶ dropout, batch normalisation

From the Maths to the Keyboard

This session is about the **choices you are about to make in the code**, and why they matter.

What the model is

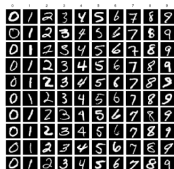
- ▶ which activation, and where
- ▶ which output layer and which loss
- ▶ normalising the inputs

How it is optimised

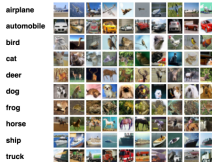
- ▶ batch, epoch, learning rate
- ▶ overfitting, and what to watch
- ▶ dropout, batch normalisation

Every line of this is a line you will type in the tutorial.

The Datasets the Field Grew Up On



(a)



(b)



(c)

A wonderful little production.

The filming technique is very unassuming- very old-time-B...

positive

I thought this was a wonderful way to spend time on a too hot summer weekend, sitting in the air con...

positive

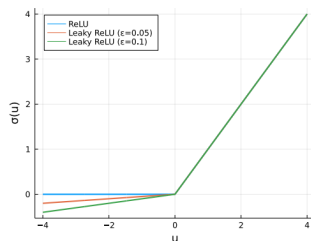
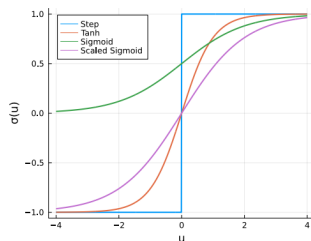
Basically there's a family where a little boy (Jake) thinks there's a zombie in his closet & his par...

negative

(d)

(a) MNIST, 28×28 digits (b) CIFAR-10, 32×32 colour images
(c) ImageNet, 1,000 classes (d) IMDB, reviews labelled positive or negative

Activation Functions

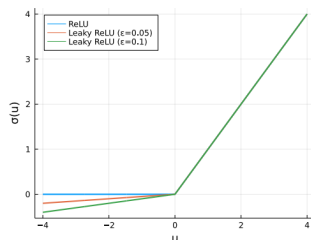
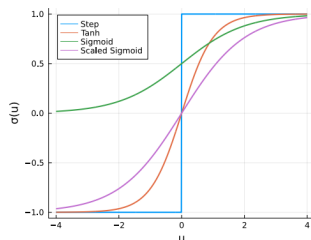


$$\sigma_{\text{Sig}}(u) = \frac{1}{1 + e^{-u}}$$

$$\sigma_{\text{Tanh}}(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}}$$

$$\sigma_{\text{ReLU}}(u) = \max(0, u)$$

Activation Functions



$$\sigma_{\text{Sig}}(u) = \frac{1}{1 + e^{-u}} \quad \sigma_{\text{Tanh}}(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}} \quad \sigma_{\text{ReLU}}(u) = \max(0, u)$$

- ▶ **Sigmoid** and **tanh** are bounded. Far from 0 the curve is flat, the gradient is nearly zero and learning stalls: the **vanishing gradient**
- ▶ **ReLU** has gradient 1 on the whole positive side. It trains far faster and is the default in hidden layers
- ▶ **Leaky ReLU**, $\max(\epsilon u, u)$, keeps a small slope on the negative side, so a unit is never dead for ever

Rule of thumb: ReLU inside; the output layer is a different question.

The Output Layer Depends on the Question

Task	Last activation	Loss
Predict a number	none (linear), 1 unit	mean squared error
Predict K numbers	none (linear), K units	mean squared error
Yes / no	sigmoid, 1 unit	binary cross-entropy
One class out of K	softmax, K units	categorical cross-entropy
Several labels at once	sigmoid, K units	binary cross-entropy

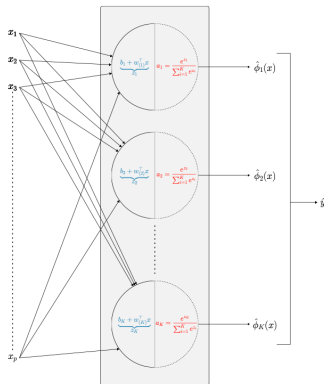
The Output Layer Depends on the Question

Task	Last activation	Loss
Predict a number	none (linear), 1 unit	mean squared error
Predict K numbers	none (linear), K units	mean squared error
Yes / no	sigmoid, 1 unit	binary cross-entropy
One class out of K	softmax, K units	categorical cross-entropy
Several labels at once	sigmoid, K units	binary cross-entropy

These lines go together. A softmax with a squared-error loss, or a sigmoid on two output units, will run without any error message and train badly.

*“Several labels at once”: a patient may have asthma and eczema and rhinitis. Each label gets its own sigmoid, because the probabilities must **not** sum to one.*

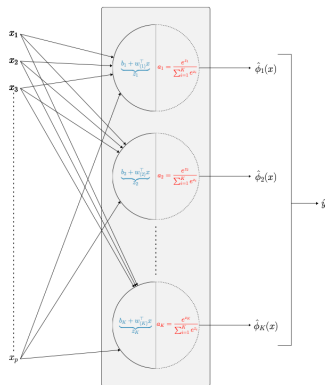
The Softmax Layer



$$a_k = \frac{e^{z_k}}{\sum_{i=1}^K e^{z_i}}, \quad k = 1, \dots, K$$

- ▶ K units, one per class, each computing its own score $z_k = b_k + w_{(k)}^T x$
- ▶ The exponential makes everything positive, the denominator makes the K outputs **sum to one**
- ▶ The classes are therefore in **competition**: raising one probability lowers the others

The Softmax Layer



$$a_k = \frac{e^{z_k}}{\sum_{i=1}^K e^{z_i}}, \quad k = 1, \dots, K$$

- ▶ K units, one per class, each computing its own score $z_k = b_k + w_{(k)}^\top x$
- ▶ The exponential makes everything positive, the denominator makes the K outputs **sum to one**
- ▶ The classes are therefore in **competition**: raising one probability lowers the others

*That competition is exactly what you do **not** want when several labels can be true at once.*

Normalising the Inputs

- ▶ A variable in years and another in ng/mL do not live on the same scale. The loss surface becomes a long narrow valley, and gradient descent zigzags across it instead of going down
- ▶ Two standard fixes, applied **column by column**:

$$\tilde{x} = \frac{x - \bar{x}}{s_x}$$

standardisation

$$\tilde{x} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

min-max to [0,1]

Normalising the Inputs

- ▶ A variable in years and another in ng/mL do not live on the same scale. The loss surface becomes a long narrow valley, and gradient descent zigzags across it instead of going down
- ▶ Two standard fixes, applied **column by column**:

$$\tilde{x} = \frac{x - \bar{x}}{s_x}$$

standardisation

$$\tilde{x} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

min-max to [0,1]

Compute $\bar{x}$ and s_x on the **training set only**, then apply *those same numbers* to the validation and test sets. Normalising everything together leaks information and flatters your results.

For images the convention is simply to divide the pixels by 255.

Batch Normalisation: Doing It Again, Inside

- ▶ The inputs were normalised once, at the entrance. After two or three layers the activations have drifted again, to a completely different scale, and the layers downstream have to chase a moving target
- ▶ **Batch normalisation** simply repeats the operation *inside* the network: at each mini-batch, the activations of a layer are recentred and rescaled

Batch Normalisation: Doing It Again, Inside

- ▶ The inputs were normalised once, at the entrance. After two or three layers the activations have drifted again, to a completely different scale, and the layers downstream have to chase a moving target
- ▶ **Batch normalisation** simply repeats the operation *inside* the network: at each mini-batch, the activations of a layer are recentred and rescaled

What you get: training is faster and much more stable, larger learning rates become possible, and the initial values of the weights matter less.

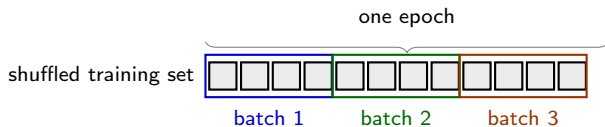
Batch Normalisation: Doing It Again, Inside

- ▶ The inputs were normalised once, at the entrance. After two or three layers the activations have drifted again, to a completely different scale, and the layers downstream have to chase a moving target
- ▶ **Batch normalisation** simply repeats the operation *inside* the network: at each mini-batch, the activations of a layer are recentred and rescaled

What you get: training is faster and much more stable, larger learning rates become possible, and the initial values of the weights matter less.

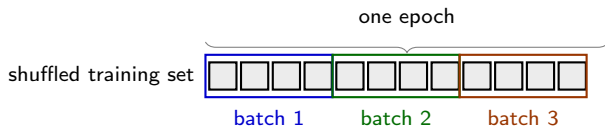
It does not behave the same at training and at prediction: at prediction it uses averages accumulated during training, not the batch.

Batch, Epoch, Iteration



$\theta(0)$ $\xrightarrow{\text{blue}}$ $\theta(1)$ $\xrightarrow{\text{green}}$ $\theta(2)$ $\xrightarrow{\text{orange}}$ $\theta(3)$ one gradient step per batch

Batch, Epoch, Iteration



$\theta(0)$ $\xrightarrow{\text{blue}}$ $\theta(1)$ $\xrightarrow{\text{green}}$ $\theta(2)$ $\xrightarrow{\text{orange}}$ $\theta(3)$ one gradient step per batch

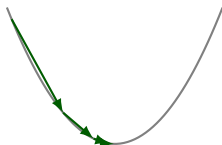
10,000 samples, `batch_size = 32` $\Rightarrow$ 313 iterations per epoch
50 epochs $\Rightarrow$ 15,650 updates in total

- ▶ The whole set at once: one clean update per epoch, slow and memory hungry. One sample at a time: very noisy
- ▶ 32 to 256 is the compromise, and it is what a GPU likes. The set is **reshuffled at every epoch**, so the batches never repeat

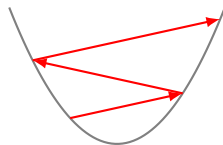
The Learning Rate: the Number That Decides



too small
it never arrives



about right
a few steps, then down

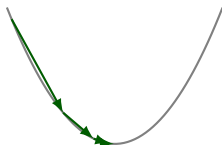


too large
it bounces and diverges

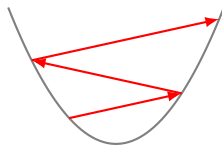
The Learning Rate: the Number That Decides



too small
it never arrives



about right
a few steps, then down

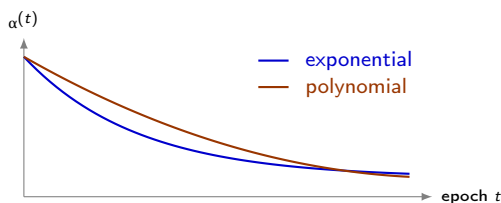


too large
it bounces and diverges

- ▶ **SGD**: follow the gradient. **Momentum**: keep some of the previous direction, which smooths the path. **Adam**: adapt the step size for each parameter separately
- ▶ Sensible defaults: **Adam** with a learning rate of 10^{-3} , or SGD with momentum at 10^{-2}
- ▶ If the loss explodes or turns into NaN, the learning rate is too large. If nothing moves for ten epochs, it is too small
- ▶ Reduce it when the validation loss stops improving

Learning Rate Decay

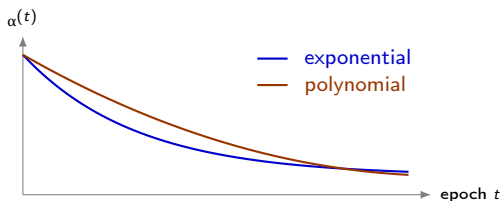
- Start **high** to cross the landscape quickly, finish **low** to settle into the minimum instead of bouncing around it



$$\alpha(t) = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) e^{-t/\tau} \quad \alpha(t) = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \left(1 - \frac{t}{T}\right)^p$$

Learning Rate Decay

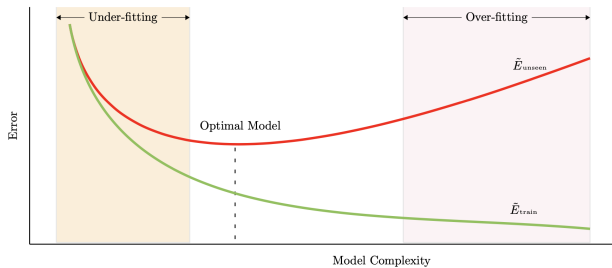
- ▶ Start **high** to cross the landscape quickly, finish **low** to settle into the minimum instead of bouncing around it



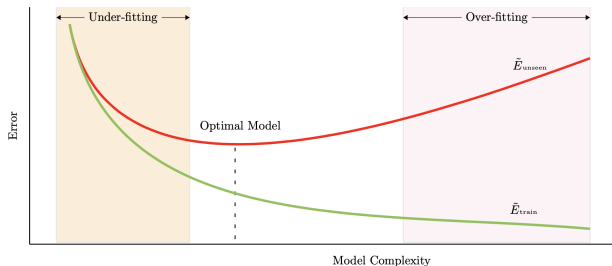
$$\alpha(t) = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) e^{-t/\tau} \quad \alpha(t) = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \left(1 - \frac{t}{T}\right)^p$$

- ▶ $\alpha_{\max}$ and the decay rate τ have to be tried out. A small $\alpha_{\min}$ keeps a little movement alive at the end
- ▶ The simplest schedule of all, and the one to start with: **divide the rate by two or by ten whenever the validation loss stops improving** (ReduceLROnPlateau)

Overfitting: the Two Curves

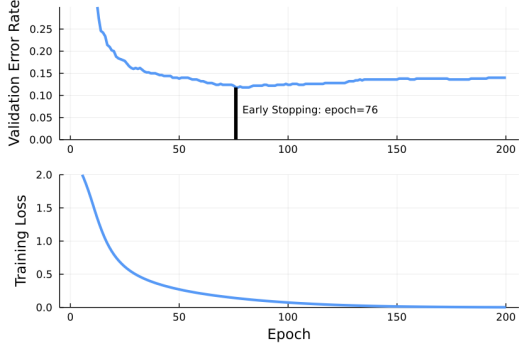


Overfitting: the Two Curves

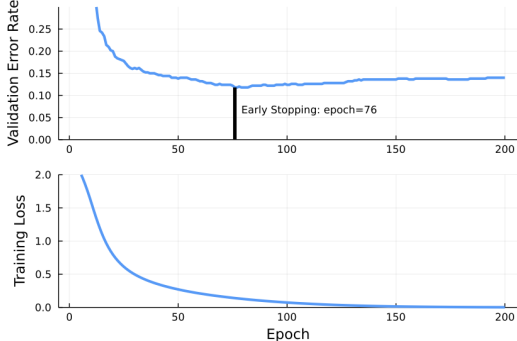


- ▶ The error on the **training** data keeps going down. The error on **unseen** data goes down, then back up
- ▶ In deep learning the same picture appears along the **epochs**: a network trained long enough will eventually memorise its training set
- ▶ So you never watch the training loss alone. You watch the **validation** loss, and a metric you actually care about

Early Stopping

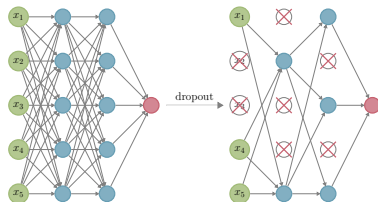


Early Stopping



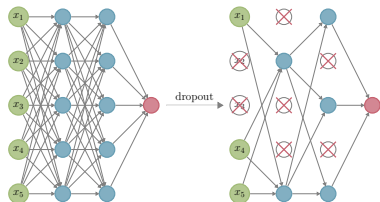
- ▶ Below, the training loss goes down for ever. Above, the validation error reaches its minimum around epoch 76, then slowly deteriorates
- ▶ Keep training while the validation error improves, stop when it has not improved for a few epochs (the **patience**), and **restore the weights of the best epoch**, not the last ones
- ▶ The cheapest regularisation there is: it costs nothing and it uses the validation set you already have

Dropout



- ▶ At each update, a fraction p of the units is switched **off at random**, along with all their connections
- ▶ No unit can rely on one particular neighbour being there, so the network is forced to **spread the information out** instead of building fragile chains
- ▶ Typical p : 0.2 to 0.5, and only in the wide layers

Dropout



- ▶ At each update, a fraction p of the units is switched **off at random**, along with all their connections
 - ▶ No unit can rely on one particular neighbour being there, so the network is forced to **spread the information out** instead of building fragile chains
 - ▶ Typical p : 0.2 to 0.5, and only in the wide layers
-
- ▶ At prediction time **everything is switched back on**, and the activations are rescaled so that the expected signal is unchanged
 - ▶ One way to read it: every update trains a slightly different network, and the final model behaves like an **average of all of them**, an **ensemble** strategy obtained for free

Weight Decay: the L_2 Penalty

Add a penalty to the objective, exactly as in ridge regression:

$$\min_{\theta} C(\theta; \mathcal{D}) + \frac{\lambda}{2} \|\theta\|^2, \quad \|\theta\|^2 = \sum_i \theta_i^2$$

- ▶ Large weights are what let a network draw wild, wiggly boundaries. Keeping them small keeps the function smooth
- ▶ The gradient step becomes

$$\theta \longleftarrow (1 - \eta\lambda)\theta - \eta \nabla C(\theta)$$

every weight is **shrunk** a little before each update: hence *weight decay*.

Weight Decay: the L_2 Penalty

Add a penalty to the objective, exactly as in ridge regression:

$$\min_{\theta} C(\theta; \mathcal{D}) + \frac{\lambda}{2} \|\theta\|^2, \quad \|\theta\|^2 = \sum_i \theta_i^2$$

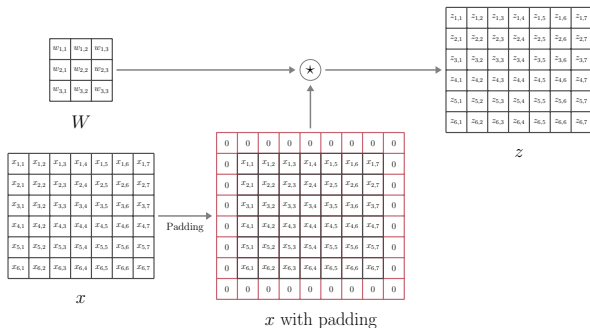
- ▶ Large weights are what let a network draw wild, wiggly boundaries. Keeping them small keeps the function smooth
- ▶ The gradient step becomes

$$\theta \leftarrow (1 - \eta\lambda)\theta - \eta \nabla C(\theta)$$

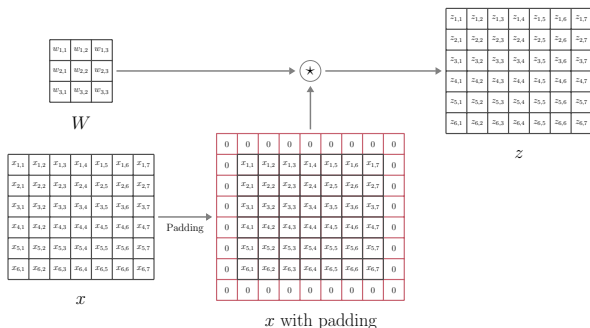
every weight is **shrunk** a little before each update: hence *weight decay*.

Typical λ : 10^{-4} to 10^{-2} . With Adam, use the AdamW variant, where the decay is applied directly to the weights.

Convolution in Practice: Padding

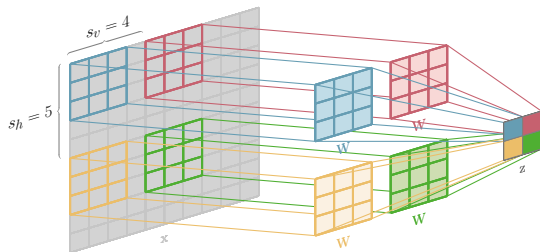


Convolution in Practice: Padding

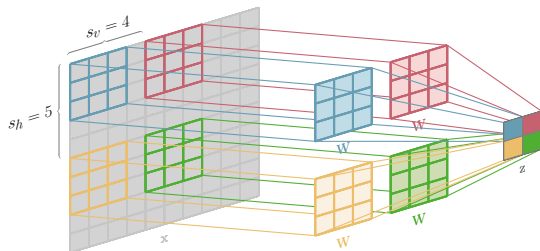


- ▶ Without padding the output is **smaller** than the input, and the pixels on the border are seen by far fewer filter positions than those in the middle
- ▶ A border of zeros fixes both: the output keeps the size of the input. In Keras this is `padding = "same"`, against `padding = "valid"` for no padding at all

Convolution in Practice: Stride

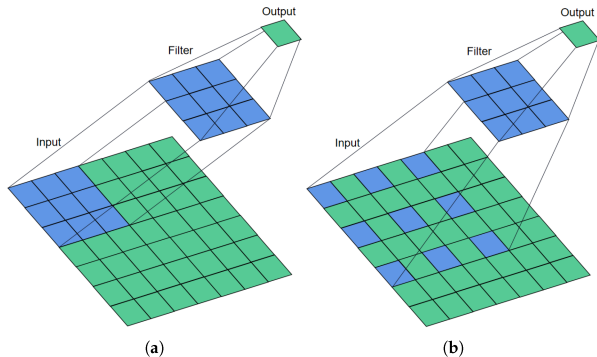


Convolution in Practice: Stride



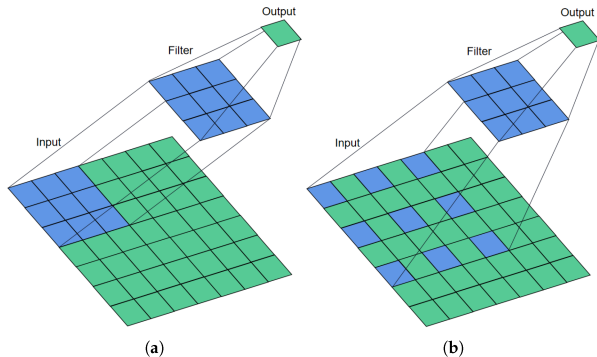
- ▶ The filter moves by s pixels at a time instead of one, so it visits fewer positions and the output shrinks by roughly a factor s
- ▶ A cheap way to reduce the resolution, and an alternative to pooling
- ▶ Padding and stride together decide the size of the output. Your framework prints it for you at every layer: read it, and check it is what you expect

Convolution in Practice: Dilation



(a) the usual filter **(b)** the same nine weights, spread out

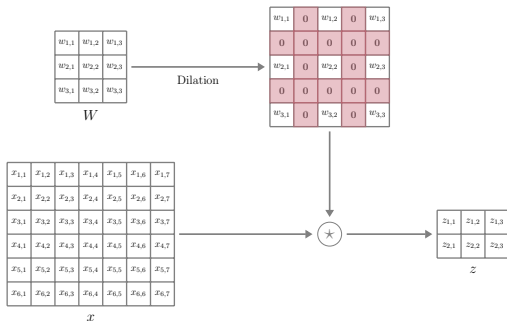
Convolution in Practice: Dilation



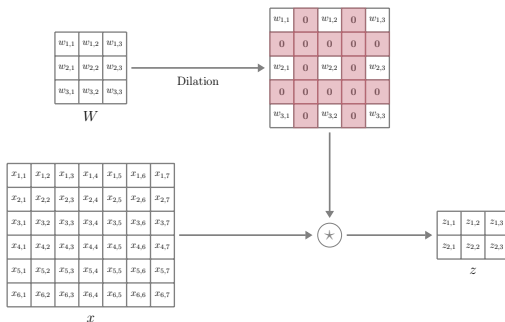
(a) the usual filter (b) the same nine weights, spread out

- ▶ Holes are inserted *inside* the filter, so the same nine weights cover a much wider area of the image
- ▶ The field of view grows **without a single extra parameter**
- ▶ Useful when the wide context matters more than the fine detail. It is rarely the first knob you touch

Dilation, Seen on the Numbers

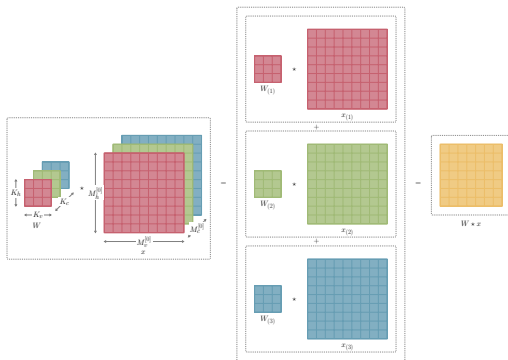


Dilation, Seen on the Numbers

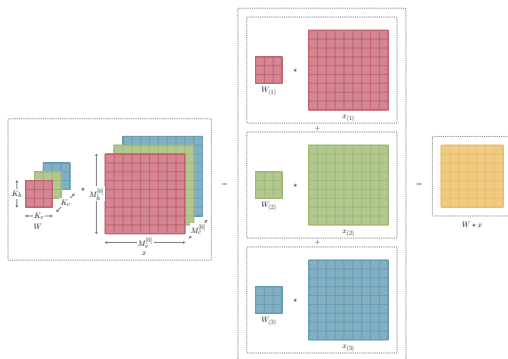


- ▶ The same 3×3 filter, with a row and a column of zeros slipped between its weights
- ▶ It now spans a 5×5 area, but only the nine original numbers are ever learned

An Input with Several Channels

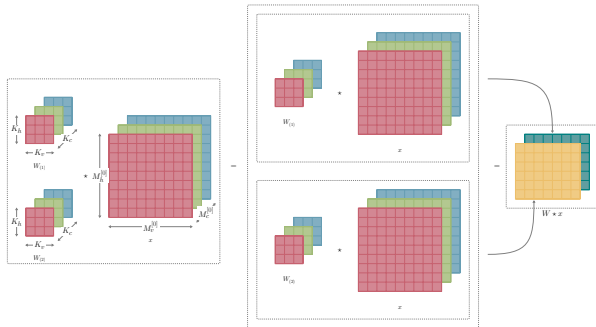


An Input with Several Channels

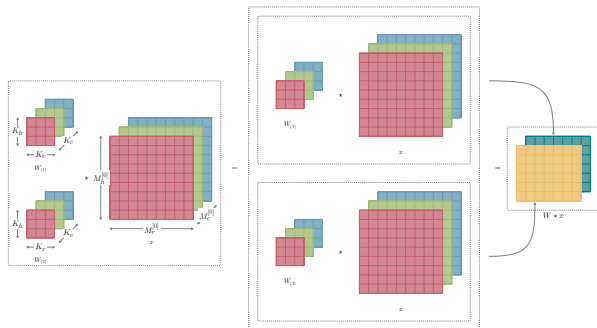


- ▶ A colour image is not a matrix but a **stack** of 3 matrices, one per channel: red, green, blue
- ▶ The filter has **as many channels as the input**. It is applied to each one, and the results are **added**
- ▶ So one filter over a 3-channel input still produces a **single** output map

Several Filters, Several Output Channels

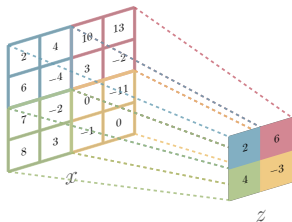
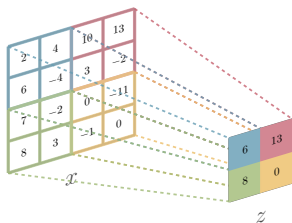


Several Filters, Several Output Channels



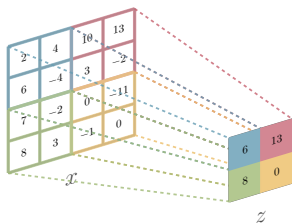
- ▶ A layer holds **several filters**, each producing its own output map. The number of filters is the number of **output channels**, and it is the number you type in the code
- ▶ With C_{in} input channels, C_{out} filters of size $K_h \times K_v$, the layer has $K_h K_v C_{in} C_{out} + C_{out}$ parameters
- ▶ A 3×3 filter on 64 channels producing 128 channels:
 $3 \cdot 3 \cdot 64 \cdot 128 + 128 = 73,856$ weights

Pooling

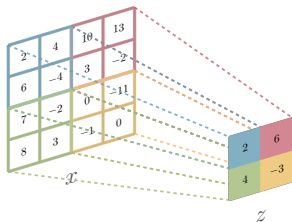


- ▶ Slide a small window and keep only **one** number per window: the largest, or the average
- ▶ The resolution drops, usually by a factor of two in each direction, so everything downstream is cheaper
- ▶ **No parameters at all**: there is nothing to learn here
- ▶ Moving the pattern by one pixel usually leaves the maximum of the window unchanged, so the output becomes a little **insensitive to small shifts**

Pooling



max pooling



average pooling

- ▶ Slide a small window and keep only **one** number per window: the largest, or the average
- ▶ The resolution drops, usually by a factor of two in each direction, so everything downstream is cheaper
- ▶ **No parameters at all**: there is nothing to learn here
- ▶ Moving the pattern by one pixel usually leaves the maximum of the window unchanged, so the output becomes a little **insensitive to small shifts**

Convolution says **how strongly the learned filter responds**, and **exactly where**. Pooling keeps the strength and gives up the exact position.

Training and Production Are Not the Same Thing

Frozen after training	Recomputed for each new sample
every weight matrix and bias	the activations of each layer
the normalisation constants	the prediction
the batch-norm running averages	

Training and Production Are Not the Same Thing

Frozen after training	Recomputed for each new sample
every weight matrix and bias	the activations of each layer
the normalisation constants	the prediction
the batch-norm running averages	

- ▶ In production **nothing is learned**. No gradient is computed, no weight moves. It is the fitted regression: $\hat{\beta}$ is fixed, only the prediction depends on the new x
- ▶ Two layers change behaviour: **dropout** is switched off, and **batch normalisation** uses its stored averages instead of the batch
- ▶ Your framework handles this for you: `model.eval()` in PyTorch, or a plain call to `predict()` in Keras

Transfer Learning

- ▶ VGG19 took days of computation on 1.5 million images. You will not repeat that, and with a few hundred images you could not
- ▶ But the first layers of a trained network detect edges, textures and shapes, and those are **not specific to the original task**

Transfer Learning

- ▶ VGG19 took days of computation on 1.5 million images. You will not repeat that, and with a few hundred images you could not
- ▶ But the first layers of a trained network detect edges, textures and shapes, and those are **not specific to the original task**

The recipe

1. take a pretrained network and **remove its last layer**;
2. **freeze** everything that remains;
3. put your own output layer on top, with your number of classes;
4. train that layer alone, then optionally **unfreeze the last block** with a *much* smaller learning rate, so that large updates do not destroy what was already learned.

This is what the tutorial does.

One Honest Word About Tabular Data

- ▶ Deep learning earned its reputation where the data has a **structure to exploit**: neighbouring pixels, successive words, consecutive time points. A convolution and an attention layer are built for exactly that
- ▶ On a **plain table** of a few thousand rows, with no such structure, a **random forest** or **gradient boosting** (XGBoost, LightGBM) is usually as accurate and trains in seconds

One Honest Word About Tabular Data

- ▶ Deep learning earned its reputation where the data has a **structure to exploit**: neighbouring pixels, successive words, consecutive time points. A convolution and an attention layer are built for exactly that
- ▶ On a **plain table** of a few thousand rows, with no such structure, a **random forest** or **gradient boosting** (XGBoost, LightGBM) is usually as accurate and trains in seconds

A reasonable strategy

- ▶ start with a penalised regression and a gradient boosting baseline;
- ▶ bring in deep learning when there is a structure it can use: images, spectra, signals, sequences, or several data types to combine in a single model;
- ▶ and always compare on the same validation split.

A deep network that does not beat a simple baseline is not a result.

Before You Hit Run

1. Are the inputs **normalised**, with the constants computed on the training set only?
2. Does the **last layer** match the question, and the **loss** match the last layer?
3. Is there a **validation set** that the model never sees during training?
4. Is the **learning rate** sensible, 10^{-3} with Adam to begin with?
5. Does the **batch size** fit in memory, 32 to 256?
6. Is **early stopping** watching the validation loss?
7. Do you have a **simple baseline** to compare against?

Before You Hit Run

1. Are the inputs **normalised**, with the constants computed on the training set only?
2. Does the **last layer** match the question, and the **loss** match the last layer?
3. Is there a **validation set** that the model never sees during training?
4. Is the **learning rate** sensible, 10^{-3} with Adam to begin with?
5. Does the **batch size** fit in memory, 32 to 256?
6. Is **early stopping** watching the validation loss?
7. Do you have a **simple baseline** to compare against?

None of these is about the architecture. They are about the data, the loss and the way you look at the results.